

Compiler Construction (CS-4031)

Sessional-II Exam

Date: 11/05/2025

Total Time: 1 hr

Course Instructors

Total Marks: 30

Shaharbano Asif, Syed Faisal Ali, Syed Zain Ul Hassan, Ansum Hamid and Zulfiqar Ali

Total Questions: 03

Roll No

Section

Student Signature

Do not write below this line.

Attempt all questions.

CLO # 1: Understand the basic techniques used in compiler construction such as lexical analysis, top-down, bottom-up parsing, context-sensitive analysis, and intermediate code generation.

Question 1: Solve the Following

(4 + 4 + 2 = 10 points)

- a) Remove the Left Recursion from the following grammar, you are required to show all steps of removing.

A → B x y | z

B → C D | Aw

C → m | n

D → p

- b) Remove ambiguity from following grammar

S → if E then S

 | if E then S else S

 | other

- c) The following grammar has multiple common prefixes. Apply left factoring to eliminate them. Show your work step by step.

<Declaration> → <Type> id ;

 | <Type> id = Expr;

<Type> → int | float | char

CLO # 2: Understand the basic data structures used in compiler construction such as abstract syntax trees, symbol tables, three-address code, and stack machines.

Question 2: Consider the following

(5 + 5 = 10 points)

<Stmt> → while (<Exp>) <Stmt>

<Stmt> → { <Stmts> }

<Stmts> → <Stmt> <Stmts>

<Stmts> → ε

<Exp> → id

- a) Create LR (0) Canonical items of the above Grammar with DFA given in Q#2.
b) Create LR (0) Parse table of the above in (Q#2a) LR (0) Canonical items, if there are conflicts, then explain conflicts, after that try it using SLR parsing table.

Question 3: Consider the following grammar

(7 +3 = 10 points)

$\langle \text{Expr} \rangle \rightarrow \langle \text{Term} \rangle \langle \text{Expr}' \rangle$
 $\langle \text{Expr}' \rangle \rightarrow \text{'or'} \langle \text{Term} \rangle \langle \text{Expr}' \rangle \mid \epsilon$
 $\langle \text{Term} \rangle \rightarrow \langle \text{Factor} \rangle \langle \text{Term}' \rangle$
 $\langle \text{Term}' \rangle \rightarrow \text{'and'} \langle \text{Factor} \rangle \langle \text{Term}' \rangle \mid \epsilon$
 $\langle \text{Factor} \rangle \rightarrow \text{'not'} \langle \text{Factor} \rangle \mid \text{'id'}$

- a) Create Recursive Descent Parser for the following Grammar, you are required to create function for all non-terminal. No need to detailed code, just write pseudo style code.
- b) Now try the above created Recursive Descent Parser in Q#3(a) with the input '**id and id**' in a tabular format.